

TechTribers

Java Interview Series · Part 8 · Q340–Q372

Modern Java 9 → 17

Release model, modules, collection factories, Stream and Optional upgrades, var, the HTTP client, text blocks, records, sealed classes and the 8 → 17 migration, release by release.

33	30	11	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `MJ-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

33	5	30	11
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🗓️	01	Release Model, Distributions & Migration	Q340–Q342	4
9️⃣	02	Java 9: Modules & API Upgrades	Q343–Q354	7
🔟	03	Java 10 & 11	Q355–Q358	17
🧪	04	Java 12 – 16	Q359–Q367	20
1️⃣7️⃣	05	Java 17 & the 8 → 17 Leap	Q368–Q372	28

© TechTribers. For personal use only. Do not redistribute.

All questions

🗓️ Release Model, Distributions & Migration

Q340 How does Java's release cadence work, and what is an LTS release? 4

Q341 What are the different JDK distributions, and is Java still free? 5

Q342 What are the main problems when migrating from Java 8 to Java 17 or later? 5

9️⃣ Java 9: Modules & API Upgrades

Q343 What is the Java Platform Module System (JPMS)? 7

Q344 What are named, automatic and unnamed modules? Do I have to use modules? 8

Q345 What is JShell, and when is it useful? 9

Q346 What do `List.of`, `Set.of` and `Map.of` return, and how do they differ from `Arrays.asList`? 9

Q347 What Stream API improvements arrived in Java 9 and later? 10

Q348 How has Optional improved since Java 8, and how should it be used? 11

Q349 What does the Java 9 Process API offer? 12

Q350 What is the Flow API (reactive streams) in Java 9? 13

Q351 What small language and API changes came in Java 9? 14

Q352 What is the StackWalker API? 15

Q353 What are VarHandles? 15

Q354 What are multi-release JARs? 16

🔟 Java 10 & 11

Q355 What is `var`? What are its rules and pitfalls? 17

Q356 What is the difference between an unmodifiable view and an unmodifiable copy? 18

Q357 How do you use the Java 11 HTTP Client? 19

Q358 What is nest-based access control (Java 11)? 20

🧪 Java 12 – 16

Q359 What does `Collectors.teeing` do? What other small APIs came in Java 12? 20

Q360 How do you format numbers like 1.2K or 3M in Java? 21

Q361 What are text blocks, and how is indentation handled? 22

Q362 What are helpful NullPointerExceptions? 23

Q363 What are records? What can and can't they do? 24

Q364 What is the difference between `Stream.toList()` and `collect(Collectors.toList())`? 25

Q365 What changed with strong encapsulation of JDK internals in Java 16 and 17? 26

Q366 What is `jpackage`, and how is it different from `jlink`? 27

Q367 What are Unix-domain socket channels (Java 16)? 27

1️⃣7️⃣ Java 17 & the 8 → 17 Leap

Q368 What are sealed classes and interfaces, and what are their rules? 28

Q369 What is the RandomGenerator API in Java 17? 29

Q370 What are deserialization filters, and what did Java 17 add? 30

Q371 What are HexFormat and always-strict floating point in Java 17? 31

Q372 What are the most important changes between Java 8 and Java 17? 31

© TechTribers. For personal use only. Do not redistribute.

Section 01

Release Model, Distributions & Migration

How Java ships today, which JDK to run, and how to move a Java 8 codebase forward safely.

Questions 340–342 · 3 questions

Q340 MJ-001

• Basic

How does Java's release cadence work, and what is an LTS release?

⚡ Quick answer

Since Java 10 a feature release ships every six months (March and September). Some releases are designated **Long-Term Support** (11, 17, 21, 25, and now every two years), which vendors patch for years; the releases in between get updates only until the next release.

What this means in practice:

- Production systems usually run an LTS (today: 21 or 25) and upgrade LTS to LTS.
- Non-LTS releases (26, 27…) are fully production quality but short-lived; teams use them to try features early.
- Features arrive as **preview** (complete but may change; needs `--enable-preview`), **incubator** (APIs in `jdk.incubator.*`) or **final**. Interview answers should say which state a feature was in for a given version.

Terminal

```
java -version
# openjdk version "25" 2025-09-16 LTS

# Use a preview feature (must match the exact JDK version)
javac --release 27 --enable-preview Main.java
java --enable-preview Main
```

Six-month feature releases since Java 10 · an LTS every two years since Java 17 (orange = LTS)



This book: 9 → 17 · Next parts: 18 → 21 and 22 → 27 (current LTS: 25 · latest: 27)

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> "Which Java version do you use and why?" is a common opener. A strong answer: "Java 21 LTS in production for virtual threads and pattern matching; evaluating 25 LTS for compact headers and the AOT cache."

🏷️ **Q341** `MJ-002` • Basic

What are the different JDK distributions, and is Java still free?

> ⚡ **Quick answer**
> All mainstream JDKs are builds of the same OpenJDK source. Free, production-ready options include Eclipse Temurin, Amazon Corretto, Azul Zulu, Microsoft Build of OpenJDK, Red Hat and Oracle OpenJDK; Oracle JDK has its own licence terms. They differ in support length, platforms and extras, not in Java itself.

Distribution	Notes
Eclipse Temurin (Adoptium)	Vendor-neutral, TCK-tested, popular default in Docker images
Amazon Corretto	Free long-term updates; default on AWS
Azul Zulu	Wide platform coverage; paid support available (Azul also sells Platform Prime)
Microsoft Build of OpenJDK	Used across Azure
Oracle OpenJDK builds	Free (GPL+CPE), updated only for six months per release
Oracle JDK	Separate licence terms that have changed over time; check before production use
GraalVM	Adds native image and the Graal JIT

> ⚠️ **Interview trap**
> "Java is no longer free" is a myth: Oracle changed the licence of **Oracle JDK**, not of Java. The same code is available from several vendors at no cost.

🧳 **Q342** `MJ-003` • Advanced

What are the main problems when migrating from Java 8 to Java 17 or later?

> ⚡ **Quick answer**
> The big breakers: Java EE and CORBA modules removed in Java 11 (JAXB, JAX-WS, `javax.annotation`), JDK internals strongly encapsulated in Java 16–17, old bytecode libraries that don't understand new class-file versions, and removed JVM flags. Upgrade build tools and libraries first, then the JDK.

© TechTribers. For personal use only. Do not redistribute.

Spring Boot 3 requires Java 17 **and** the `javax.` → `jakarta.` namespace change, so the JDK upgrade and the Spring Boot 3 upgrade are often done together. Migrate to Java 17 on Spring Boot 2.7 first, then move to Boot 3, so the two changes aren't tangled.

Terminal

```
# 1. Find use of JDK-internal APIs (sun.misc, com.sun.*)
jdeps --jdk-internals --multi-release 17 -cp 'lib/*' app.jar

# 2. Find deprecated-for-removal APIs
jdeprscan --release 17 app.jar

# 3. Replace removed Java EE APIs with Maven dependencies
#    javax.xml.bind:jaxb-api      ->  jakarta.xml.bind:jakarta.xml.bind-api + runtime
#    javax.annotation.PostConstruct -> jakarta.annotation:jakarta.annotation-api

# 4. Last-resort workaround for reflective access (document every one)
java --add-opens java.base/java.lang=ALL-UNNAMED -jar app.jar
```



- ✓ jdeps --jdk-internals: find internal API use
- ✓ upgrade Lombok, Mockito, ByteBuddy, ASM, Spring
- ✓ rerun GC tuning: CMS & PermGen flags are gone
- ✓ add jakarta.xml.bind / jaxws deps if needed
- ✓ remove --illegal-access, use --add-opens sparingly

> 💡 **Pro tip**
> Typical stragglers are Lombok, Mockito, ByteBuddy, ASM, Jacoco, old Hibernate and Groovy versions. Each needs a version that knows the new class-file format before the build will even pass.

© TechTribers. For personal use only. Do not redistribute.

Section 02

Java 9: Modules & API Upgrades

The module system and the many smaller APIs that arrived with it: factories, stream and Optional upgrades, processes, reactive streams.

Questions 343–354 · 12 questions

Q343 MJ-004 • Advanced

What is the Java Platform Module System (JPMS)?

⚡ Quick answer

JPMS (Java 9, Project Jigsaw) groups packages into **modules** declared in `module-info.java`. A module states what it `requires` and which packages it `exports`; everything else is encapsulated, even `public` classes. The JDK itself was split into about 70 modules.

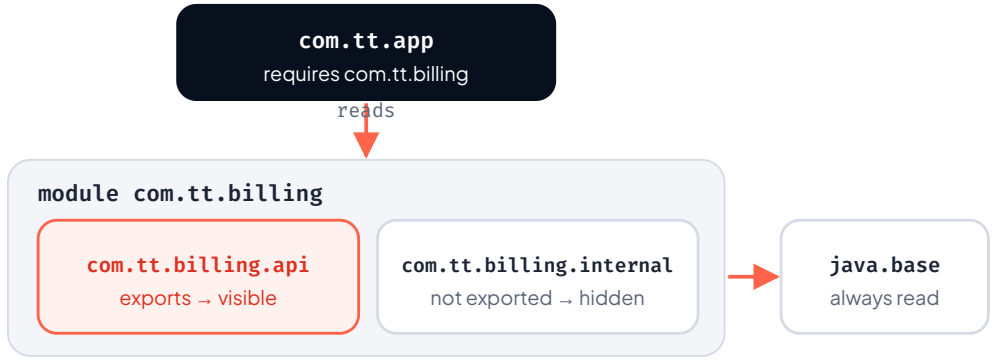
Benefits:

- **Reliable configuration**: missing modules fail at start-up, not with a `NoClassDefFoundError` an hour later.
- **Strong encapsulation**: internal packages are invisible at compile time and runtime.
- **Smaller runtimes**: `jlink` builds a custom JRE containing only the modules you use.

Java

```
// src/com.tt.billing/module-info.java
module com.tt.billing {
    requires java.sql;                          // dependency
    requires transitive com.tt.common;          // callers get it too
    exports com.tt.billing.api;                 // public API
    exports com.tt.billing.spi to com.tt.app;   // qualified export
    opens com.tt.billing.model to com.fasterxml.jackson.databind;   // deep reflection
    provides com.tt.billing.api.TaxRule with com.tt.billing.internal.GstRule;
    uses com.tt.billing.api.Exporter;           // ServiceLoader consumer
}
```


© TechTribers. For personal use only. Do not redistribute.



Even public classes in non-exported packages are inaccessible: strong encapsulation at compile time AND runtime.

⚠️ **Interview trap**

`exports` grants compile-time and normal reflective access to public types; frameworks that reflect on private fields (Jackson, Hibernate) also need `opens`.

Q344 MJ-005 • Advanced

What are named, automatic and unnamed modules? Do I have to use modules?

⚡ **Quick answer**

No. Code on the **classpath** goes into the unnamed module and works as before. A plain JAR on the **module path** becomes an automatic module. Only JARs with a `module-info.class` are fully named modules. Most applications still run on the classpath.

Practical guidance:

- Libraries should at least add an `Automatic-Module-Name` to their manifest so modular apps get a stable name.
- Full modularisation pays off for large codebases, platform teams, and `jlink`-based small runtimes.
- Spring Boot fat JARs run on the classpath; strong encapsulation of the **JDK** still applies to them.

Named module	Automatic module	Unnamed module
has module-info.java	plain JAR on module path	everything on the classpath
explicit requires/exports	name from Automatic-Module-Name or file; exports all	reads all modules; what most apps still use

⚠️ **Interview trap**

Split packages (the same package in two modules) are illegal on the module path. Old libraries that do this force you to stay on the classpath.

© TechTribers. For personal use only. Do not redistribute.

Q345 MJ-006

• Basic

What is JShell, and when is it useful?

⚡ Quick answer

JShell (Java 9) is the Java REPL: type expressions, statements or whole methods and see results immediately, without a class or `main`. It's ideal for exploring APIs, testing regexes and date formats, and teaching.

Terminal

```
$ jshell
jshell> var nums = List.of(5, 3, 8)
nums ==> [5, 3, 8]

jshell> nums.stream().mapToInt(i -> i).summaryStatistics()
$2 ==> IntSummaryStatistics{count=3, sum=16, min=3, average=5.333333, max=8}

jshell> "2026-10-14".matches("\\d{4}-\\d{2}-\\d{2}")
$3 ==> true

jshell> /imports      // list imports    /vars    /methods    /save session.jsh    /exit
```

💡 Pro tip

Combine with Java 11+ single-file launch (`java Script.java`) and Java 25's compact source files for quick scripts without a build tool.

Q346 MJ-007

• Intermediate

What do `List.of`, `Set.of` and `Map.of` return, and how do they differ from `Arrays.asList`?

⚡ Quick answer

The Java 9 factories return **unmodifiable** collections that reject `null` and (for `Set` / `Map`) duplicate keys at creation, with deliberately unspecified iteration order for sets and maps. `Arrays.asList` returns a fixed-size view backed by the array: you can `set` elements but not add or remove.

Java

```
List<String> a = List.of("java", "kafka");
List<String> b = Arrays.asList("java", "kafka");

try { a.set(0, "x"); } catch (UnsupportedOperationException e) { System.out.println("List.of: fully
unmodifiable"); }
b.set(0, "spring");                                    // allowed
System.out.println("asList after set: " + b);
try { b.add("x"); } catch (UnsupportedOperationException e) { System.out.println("asList: fixed
```


© TechTribers. For personal use only. Do not redistribute.

```
size"); }

try { List.of("a", null); } catch (NullPointerException e) { System.out.println("List.of rejects
null"); }
try { Set.of("x", "x"); } catch (IllegalArgumentException e) { System.out.println("Set.of rejects
duplicates"); }
System.out.println(Map.ofEntries(Map.entry("k1", 1), Map.entry("k2", 2)).size());
```

▶ Output

```
List.of: fully unmodifiable
asList after set: [spring, kafka]
asList: fixed size
List.of rejects null
Set.of rejects duplicates
2
```

⚠️ **Interview trap**

`list.contains(null)` on a `List.of(...)` throws `NullPointerException` instead of returning `false`. Code ported from `ArrayList` can break on this.

Q347 MJ-008 • Intermediate

What Stream API improvements arrived in Java 9 and later?

⚡ **Quick answer**

Java 9 added `takeWhile`, `dropWhile`, a three-argument `iterate(seed, hasNext, next)` and `Stream.ofNullable`. Later: `Collectors.teeing` (12), `Stream.toList()` and `mapMulti` (16), and stream gatherers (24).

Java

```
List<Integer> sorted = List.of(1, 3, 5, 8, 9, 11);
System.out.println(sorted.stream().takeWhile(n -> n % 2 == 1).toList());     // stop at first even
System.out.println(sorted.stream().dropWhile(n -> n % 2 == 1).toList());     // skip until first
even

System.out.println(Stream.iterate(1, n -> n <= 100, n -> n * 3).toList());    // like a for loop

String maybe = null;
System.out.println(Stream.ofNullable(maybe).count() + " " + Stream.ofNullable("x").count());

// mapMulti (16): emit zero or more elements per input, without creating inner streams
List<Object> mixed = List.of(1, List.of(2, 3), 4);
List<Integer> flat = mixed.stream().<Integer>mapMulti((o, sink) -> {
    if (o instanceof Integer i) sink.accept(i);
    else if (o instanceof List<?> l) l.forEach(x -> sink.accept((Integer) x));
}).toList();
System.out.println(flat);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
[1, 3, 5]
[8, 9, 11]
[1, 3, 9, 27, 81]
0 1
[1, 2, 3, 4]
```

⚠️ Interview trap

`takeWhile` / `dropWhile` stop at the **first** element that fails the predicate. On unordered data they're not filters; use `filter` if you mean "all matching elements".

Q348 MJ-009 • Intermediate

How has Optional improved since Java 8, and how should it be used?

⚡ Quick answer

Java 9 added `ifPresentOrElse`, `or` and `stream`; Java 10 added the no-argument `orElseThrow()` (the clearer alias of `get()`); Java 11 added `isEmpty`. Use `Optional` as a **return type** for "maybe no result", not for fields, parameters or collections.

Java

```
record User(String name, String email) { }
static final Map<Integer, User> DB = Map.of(1, new User("Asha", "asha@tt.com"));
static Optional<User> find(int id) { return Optional.ofNullable(DB.get(id)); }

find(1).ifPresentOrElse(u -> System.out.println("found " + u.name()),
                        () -> System.out.println("not found"));
find(2).ifPresentOrElse(u -> System.out.println("found " + u.name()),
                        () -> System.out.println("not found"));

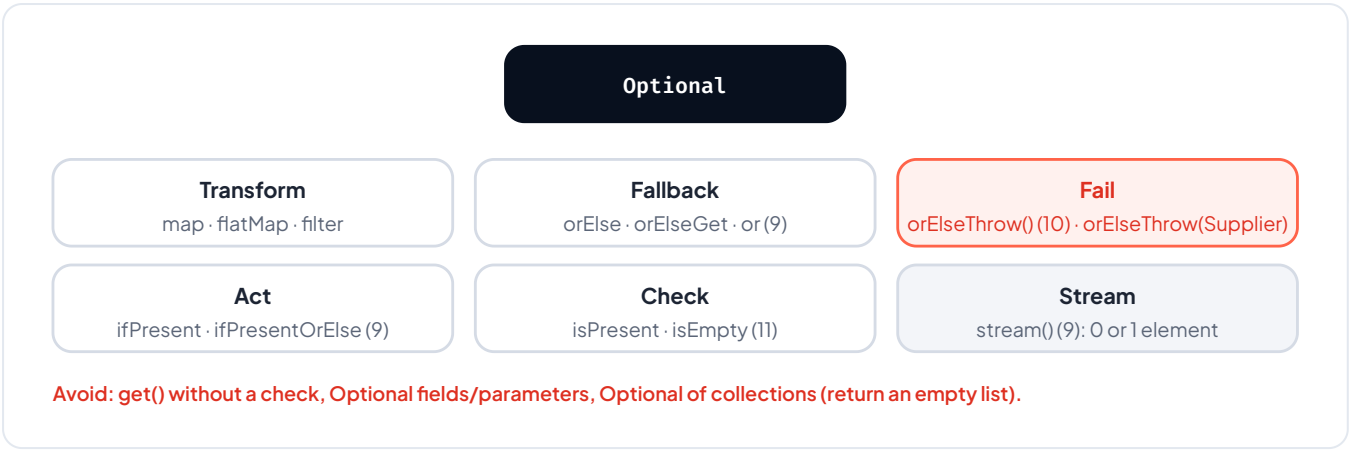
String email = find(2).or(() -> find(1)).map(User::email).orElse("none");   // or(): lazy fallback Optional
System.out.println(email + " " + find(2).isEmpty());

List<String> names = Stream.of(1, 2, 3).map(id ->
find(id)).flatMap(Optional::stream).map(User::name).toList();
System.out.println(names);
```

▶ Output

```
found Asha
not found
asha@tt.com true
[Asha]
```


© TechTribers. For personal use only. Do not redistribute.



⚠️ **Interview trap**

`orElse(expensiveCall())` **always** evaluates the argument, even when a value is present. Use `orElseGet(() -> expensiveCall())` for anything costly or side-effecting.

⚙️ **Q349** MJ-010 • Intermediate

What does the Java 9 Process API offer?

⚡ **Quick answer**

`ProcessHandle` gives cross-platform access to process IDs, parent/child processes, command lines, start times and CPU usage, plus `onExit()` as a `CompletableFuture`. Before Java 9 you had to parse `ps` output or use JNI.

Java

```
ProcessHandle self = ProcessHandle.current();
System.out.println("my pid > 0: " + (self.pid() > 0));
System.out.println("has info: " + self.info().command().isPresent());

Process p = new ProcessBuilder("java", "-version").redirectErrorStream(true).start();
String firstLine = new String(p.getInputStream().readAllBytes()).lines().findFirst().orElse("");
p.onExit().thenAccept(ph -> System.out.println("child exited: " + ph.exitValue())).join();
System.out.println(firstLine.contains("version"));
```

▶ **Output**

```
my pid > 0: true
has info: true
child exited: 0
true
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> `ProcessHandle.allProcesses()` and `self.descendants()` let a supervisor find and stop child processes it started, which is useful for build tools and test harnesses that must not leak processes.

📡 **Q350** `MJ-011` • Advanced

What is the Flow API (reactive streams) in Java 9?

> ⚡ **Quick answer**
> `java.util.concurrent.Flow` standardises the Reactive Streams interfaces (`Publisher` , `Subscriber` , `Subscription` , `Processor`) for asynchronous streams with **back-pressure**: the subscriber requests how many items it can handle. `SubmissionPublisher` is the JDK's ready-made publisher.

You rarely implement these interfaces yourself: Project Reactor (Spring WebFlux), RxJava and Mutiny implement them and interoperate through `Flow` adapters. Since virtual threads, many teams choose simple blocking code over reactive pipelines for typical I/O services.

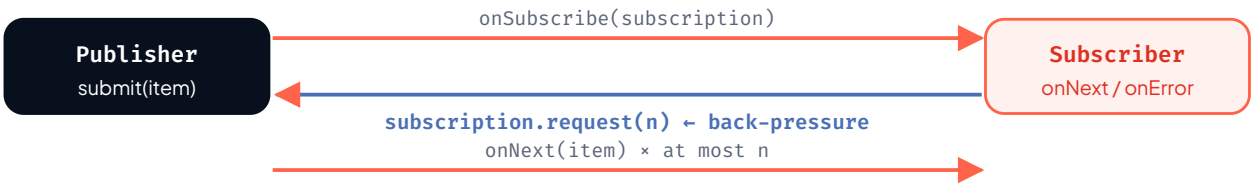
Java

```
class PrintSubscriber implements Flow.Subscriber<String> {
    private Flow.Subscription sub;
    final CountDownLatch done = new CountDownLatch(1);
    public void onSubscribe(Flow.Subscription s) { sub = s; s.request(1); }        // back-pressure:
1 at a time
    public void onNext(String item) { System.out.println("got " + item); sub.request(1); }
    public void onError(Throwable t) { t.printStackTrace(); done.countDown(); }
    public void onComplete() { System.out.println("complete"); done.countDown(); }
}
PrintSubscriber s = new PrintSubscriber();
try (SubmissionPublisher<String> pub = new SubmissionPublisher<>()) {
    pub.subscribe(s);
    List.of("order-1", "order-2").forEach(pub::submit);
}                                                  // close() -> onComplete
s.done.await();
```

▶ **Output**

```
got order-1
got order-2
complete
```


© TechTribers. For personal use only. Do not redistribute.



java.util.concurrent.Flow defines the interfaces; Reactor, RxJava, Akka Streams and Mutiny implement them.

Q351 MJ-012

• Intermediate

What small language and API changes came in Java 9?

⚡ Quick answer

Private methods in interfaces, effectively-final resources in try-with-resources, the diamond operator with anonymous classes, `@SafeVarargs` on private methods, underscore no longer a valid identifier, plus APIs like `Objects.requireNonNullElse`, `StackWalker` and `CompletableFuture.orTimeout`.

Java

```
interface Greeter {
    default String hello(String n) { return wrap("Hello " + n); }
    default String bye(String n)   { return wrap("Bye " + n); }
    private String wrap(String s)  { return "[" + s + "]"; }        // private helper (Java 9)
}
BufferedReader reader = new BufferedReader(new StringReader("line-1"));
try (reader) {                                                      // effectively final resource
(Java 9)
    System.out.println(reader.readLine());
}
Comparator<String> byLen = new Comparator<>() {                     // diamond + anonymous class
(Java 9)
    public int compare(String a, String b) { return Integer.compare(a.length(), b.length()); }
};
System.out.println(new Greeter() { }.hello("Asha") + " " + byLen.compare("ab", "abc"));
System.out.println(Objects.requireNonNullElse(null, "default"));

String result = CompletableFuture.supplyAsync(() -> { sleep(500); return "slow"; })
        .completeOnTimeout("cached", 50, TimeUnit.MILLISECONDS).join();
System.out.println(result);

static void sleep(long ms) { try { Thread.sleep(ms); } catch (InterruptedException e) { throw new
RuntimeException(e); } }
```

▶ Output

```
line-1
[Hello Asha] -1
```


© TechTribers. For personal use only. Do not redistribute.

```
default
cached
```

Q352 MJ-013 • Advanced

What is the StackWalker API?

⚡ Quick answer

`StackWalker` (Java 9) walks the current thread's stack lazily as a `Stream` of frames, so you can find the caller or filter frames without capturing the whole stack trace like `new Throwable().getStackTrace()` does. It's much cheaper and can expose the declaring `Class`.

Logging frameworks (Log4j 2, Logback) use it to compute caller location, and security-sensitive libraries use `getCallerClass()` to check who is calling them.

Java

```java
static String caller() {
    return StackWalker.getInstance()
            .walk(frames -> frames.skip(2).findFirst())         // 0 = caller(), 1 = log(), 2 = who called log
            .map(f -> f.getMethodName() + ":" + f.getLineNumber())
            .orElse("?");
}
static void log(String msg) { System.out.println(caller().split(":")[0] + " -> " + msg); }
static void placeOrder() { log("order placed"); }

placeOrder();
Class<?> who = StackWalker.getInstance(StackWalker.Option.RETAIN_CLASS_REFERENCE).getCallerClass();
System.out.println(who == null ? "no caller" : "caller class found");
```

▶ Output

```
placeOrder -> order placed
caller class found
```

Q353 MJ-014 • Advanced

What are VarHandles?

⚡ Quick answer

`VarHandle` (Java 9) is a typed, safe reference to a field or array element that supports atomic and memory-ordering operations: `compareAndSet`, `getAndAdd`, `getAcquire` / `setRelease` and more. It replaced most uses of the internal `sun.misc.Unsafe` in the JDK and in libraries.

© TechTribers. For personal use only. Do not redistribute.

```java
class Counter {
    private volatile int count;
    private static final VarHandle COUNT;
    static {
        try { COUNT = MethodHandles.lookup().findVarHandle(Counter.class, "count", int.class); }
        catch (ReflectiveOperationException e) { throw new ExceptionInInitializerError(e); }
    }
    int increment() { return (int) COUNT.getAndAdd(this, 1) + 1; }
    boolean resetIf(int expected) { return COUNT.compareAndSet(this, expected, 0); }
}
Counter c = new Counter();
for (int i = 0; i < 5; i++) c.increment();
System.out.println(c.increment() + " " + c.resetIf(6) + " " + c.resetIf(6));
```

▶ Output

```
6 true false
```

💡 **Pro tip**

Application code rarely needs VarHandles: `AtomicInteger` and friends are built on them. They matter when you need atomic updates on a field of an existing class without wrapping it, or weaker memory modes for high-performance libraries.

📦 **Q354** MJ-015 • Intermediate

What are multi-release JARs?

⚡ **Quick answer**

A multi-release JAR (Java 9) contains version-specific classes under `META-INF/versions/N/`, and the JVM picks the newest version that matches the running JDK. Libraries can use new APIs on new JDKs while still supporting old ones from a single artifact.

Real examples: Jackson, Log4j 2 and Byte Buddy ship multi-release JARs. Tools such as `jdeps` need `--multi-release <N>` to analyse them correctly.

```terminal
app.jar
├── META-INF/MANIFEST.MF            # Multi-Release: true
├── com/tt/util/Hasher.class        # baseline: Java 8 implementation
└── META-INF/versions/
    ├── 11/com/tt/util/Hasher.class # used on Java 11-20
    └── 21/com/tt/util/Hasher.class # used on Java 21+

# Build with: jar --create --file app.jar -C classes . --release 21 -C classes21 .
```


© TechTribers. For personal use only. Do not redistribute.

Section 03

Java 10 & 11

Local type inference, true immutable copies, the standard HTTP client and nest-based access.

Questions 355–358 · 4 questions



Q355 MJ-016

• Intermediate

What is `var`? What are its rules and pitfalls?

⚡ Quick answer

`var` (Java 10) asks the compiler to infer a **local variable's** type from its initializer. It's still static typing: the type is fixed at compile time. It can't be used for fields, method parameters or return types, or without an initializer.

Not allowed: `var x;` (no initializer), `var x = null;`, `var arr = {1, 2};` (array initializer), `var f = () -> 1;` (lambda needs a target type), fields, parameters, return types.

Java

```
var orders = new ArrayList<Map<String, List<Integer>>>();     // removes noise
for (var entry : Map.of("a", 1).entrySet()) System.out.println(entry.getKey() + "=" +
entry.getValue());

var list = new ArrayList<>();                 // pitfall: ArrayList<Object>
list.add("x"); list.add(42);
System.out.println(list);

var n = 10;                                   // int, not long
var total = 10L * n;                          // long
var anon = new Object() { int hits = 3; };    // var can hold an anonymous type
System.out.println(total + " " + anon.hits);

BiFunction<Integer, Integer, Integer> add = (var a, var b) -> a + b;    // Java 11: var in lambdas
System.out.println(add.apply(2, 3));
```

▶ Output

```
a=1
[x, 42]
100 3
5
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

Use `var` when the type is obvious from the right-hand side (`new` , factory methods). Avoid it when the initializer hides the type (`var result = service.process(x);`), because readers lose information.

Q356 MJ-017 • Intermediate

What is the difference between an unmodifiable view and an unmodifiable copy?

⚡ **Quick answer**

`Collections.unmodifiableList(list)` is a read-only **view**: changes to the original list show through. `List.copyOf(list)` (Java 10) and `Collectors.toUnmodifiableList()` create an independent **copy** that can never change.

Java

```
List<String> source = new ArrayList<>(List.of("java"));
List<String> view = Collections.unmodifiableList(source);
List<String> copy = List.copyOf(source);

source.add("kafka");
System.out.println("view: " + view + "  copy: " + copy);

List<String> already = List.of("a");
System.out.println("copyOf reuses immutable input: " + (List.copyOf(already) == already));
System.out.println(Stream.of("x",
"y").collect(Collectors.toUnmodifiableList()).getClass().getSimpleName().startsWith("List"));
```

▶ **Output**

```
view: [java, kafka]  copy: [java]
copyOf reuses immutable input: true
true
```

⚠️ **Interview trap**

Returning `Collections.unmodifiableList(internalList)` from a getter still exposes later internal changes to callers, and can throw `ConcurrentModificationException` while they iterate. For true immutability, copy.

© TechTribers. For personal use only. Do not redistribute.

Q357 MJ-018 • Intermediate

How do you use the Java 11 HTTP Client?

⚡ **Quick answer**

`java.net.http.HttpClient` (standard since Java 11) supports HTTP/1.1 and HTTP/2, synchronous `send` and asynchronous `sendAsync`, timeouts, redirects, authentication and WebSockets. Build one client and reuse it; requests are immutable.

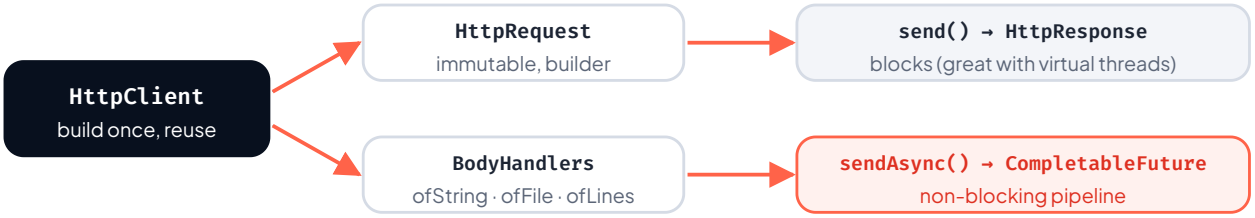
Java

```
HttpClient client = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_2)
        .connectTimeout(Duration.ofSeconds(3))
        .followRedirects(HttpClient.Redirect.NORMAL)
        .build();

HttpRequest req = HttpRequest.newBuilder(URI.create("https://api.techtriber.com/orders/42"))
        .header("Accept", "application/json")
        .timeout(Duration.ofSeconds(5))               // per-request timeout
        .GET().build();

HttpResponse<String> res = client.send(req, HttpResponse.BodyHandlers.ofString());   // blocking
System.out.println(res.statusCode() + " " + res.body());

client.sendAsync(req, HttpResponse.BodyHandlers.ofString())                          // non-
blocking
      .thenApply(HttpResponse::body)
      .thenAccept(System.out::println)
      .join();
```



HTTP/1.1 and HTTP/2 (Java 11) · HTTP/3 support added in JDK 26 · WebSocket API included

⚠️ **Interview trap**

Without `.timeout(...)` on the request, a hung server can block a thread forever. Always set both connect and request timeouts.

💡 **Pro tip**

In Spring Boot 3.2+, `RestClient` and `JdkClientHttpRequestFactory` can use this client under the hood, so you get HTTP/2 without extra dependencies.

© TechTribers. For personal use only. Do not redistribute.

Q358 MJ-019

• Advanced

What is nest-based access control (Java 11)?

⚡ Quick answer

Classes compiled from the same outer class (the outer class and its nested classes) now form a **nest** and may access each other's private members directly at the JVM level. Before Java 11, the compiler generated hidden "bridge" accessor methods (`access$000`) to make this work.

Why it matters: no extra synthetic methods in stack traces or the method count, and reflection now agrees with the language: `setAccessible` is no longer needed to reach a nestmate's private field via `Field.get` .

Java

```
public class Outer {
    private int secret = 42;
    class Inner { int read() { return secret; } }      // private access across nest mates
}
// Java 8 bytecode: Outer gets a synthetic static method  access$000(Outer)
// Java 11+:        Outer and Outer$Inner are nestmates -> direct getfield, no bridge

System.out.println(Outer.class.isNestmateOf(Outer.Inner.class));
System.out.println(Outer.Inner.class.getNestHost().getSimpleName());
```

▶ Output

```
true
Outer
```

Section 04

Java 12 – 16

Text blocks, helpful NPEs, records, Stream.toList, new collectors and stronger encapsulation of JDK internals.

Questions 359–367 · 9 questions



Q359 MJ-020

• Intermediate

What does `Collectors.teeing` do? What other small APIs came in Java 12?

⚡ Quick answer

`Collectors.teeing(c1, c2, merger)` sends every element to two collectors in one pass and merges their results. Java 12 also added `String.indent` , `String.transform` , `Files.mismatch` and compact number formatting.

© TechTribers. For personal use only. Do not redistribute.

```java
record Order(String id, double amount) { }
List<Order> orders = List.of(new Order("A", 1200), new Order("B", 450), new Order("C", 800));

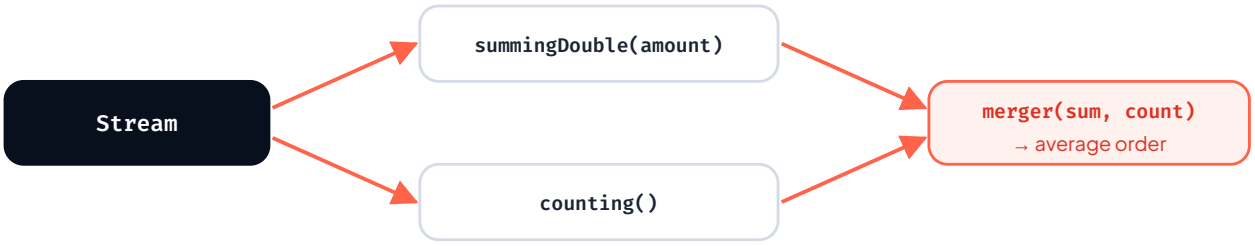
double avg = orders.stream().collect(Collectors.teeing(
        Collectors.summingDouble(Order::amount),
        Collectors.counting(),
        (sum, count) -> sum / count));
System.out.printf("average = %.2f%n", avg);

var minMax = orders.stream().collect(Collectors.teeing(
        Collectors.minBy(Comparator.comparingDouble(Order::amount)),
        Collectors.maxBy(Comparator.comparingDouble(Order::amount)),
        (min, max) -> min.get().id() + ".." + max.get().id()));
System.out.println(minMax);

String shout = "java".transform(s -> s.toUpperCase() + "!");
System.out.println(shout);
System.out.print("line1\nline2".indent(4));
```

▶ Output

```
average = 816.67
B..A
JAVA!
    line1
    line2
```



Collectors.teeing (Java 12): one pass, two collectors, one combined result.

Q360 MJ-021 • Basic

How do you format numbers like 1.2K or 3M in Java?

⚡ Quick answer

Use `NumberFormat.getCompactNumberInstance(locale, style)` (Java 12). It produces locale-aware short or long forms such as "1K", "1.5 million" or "10 lakh", with configurable fraction digits.

© TechTribers. For personal use only. Do not redistribute.

```java
NumberFormat shortUs = NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.SHORT);
shortUs.setMaximumFractionDigits(1);
System.out.println(shortUs.format(1_234) + " " + shortUs.format(23_500) + " " +
shortUs.format(4_200_000));

NumberFormat longUs = NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.LONG);
System.out.println(longUs.format(2_000_000));

NumberFormat india = NumberFormat.getCompactNumberInstance(Locale.forLanguageTag("en-IN"),
NumberFormat.Style.SHORT);
System.out.println(india.format(1_500_000));
```

▶ Output

```
1.2K 23.5K 4.2M
2 million
15L
```

💡 **Pro tip**

Perfect for dashboards and social-style counters ("23.5K followers"). Exact output depends on the JDK's CLDR locale data, so test the locales you ship.

Q361 MJ-022 • Basic

What are text blocks, and how is indentation handled?

⚡ **Quick answer**

Text blocks (final in Java 15) are multi-line string literals delimited by `"""`. The compiler strips **incidental** indentation (the common leading whitespace, using the closing delimiter as a margin), normalises line endings to `\n`, and removes trailing spaces.

```java
String json = """
        {
          "lang": "Java",
          "lts": 25
        }
        """;
System.out.print(json);

String sql = """
        SELECT id, name \
        FROM users \
        WHERE active = true""";           // \<newline> joins lines; no trailing newline
System.out.println(sql);

String formatted = """
```


© TechTribers. For personal use only. Do not redistribute.

```
        Hello %s, you have %d new orders
        """.formatted("Asha", 3);
System.out.print(formatted);
```

▶ Output

```
{
  "lang": "Java",
  "lts": 25
}
SELECT id, name FROM users WHERE active = true
Hello Asha, you have 3 new orders
```

Text block: incidental indentation (dots) is stripped using the closing delimiter as the margin

```
String json = """
        {
        "lang": "Java",
        "lts": 25
        }
        """;
```

→

```
{
"lang": "Java",
"lts": 25
}
```

Escapes: a backslash at line end joins lines · \s keeps a trailing space · other trailing spaces are stripped

⚠️ **Interview trap**

Text blocks are not templates. Building SQL with `.formatted(userInput)` is still SQL injection; use `PreparedStatement` parameters.

💡 **Q362** MJ-023 • Basic

What are helpful NullPointerExceptions?

⚡ **Quick answer**

Since Java 14 (on by default from 15), the JVM's NPE message names exactly which expression was null, e.g. `Cannot invoke "String.length()" because "<local1>.address.city" is null`, instead of just a line number.

Java

```
record Address(String city) { }
record Customer(String name, Address address) { }

Customer c = new Customer("Asha", new Address(null));
try {
    int len = c.address().city().length();
} catch (NullPointerException e) {
```


© TechTribers. For personal use only. Do not redistribute.

```
    System.out.println(e.getMessage());
}
```

▶ Output

```
Cannot invoke "String.length()" because the return value of "Address.city()" is null
```

💡 Pro tip

Local variable names appear in messages only when classes are compiled with `-g` (debug info); otherwise you see `<local4>`. Most build tools include it by default.

Q363 MJ-024 • Intermediate

What are records? What can and can't they do?

⚡ Quick answer

Records (final in Java 16) are transparent, shallowly immutable data carriers: the compiler generates private final fields, a canonical constructor, accessors, `equals`, `hashCode` and `toString` from the component list. Validation goes in a **compact constructor**.

Limitations to state in interviews:

- No extra instance fields, no `extends` (they extend `java.lang.Record`), implicitly `final`.
- JPA entities can't be records (entities need mutability, a no-arg constructor and proxies), but records work well as DTOs and projections.
- Jackson 2.12+ and Spring support records natively.

Java

```
record Money(String currency, long paise) implements Comparable<Money> {
    static final Money ZERO_INR = new Money("INR", 0);             // statics are allowed

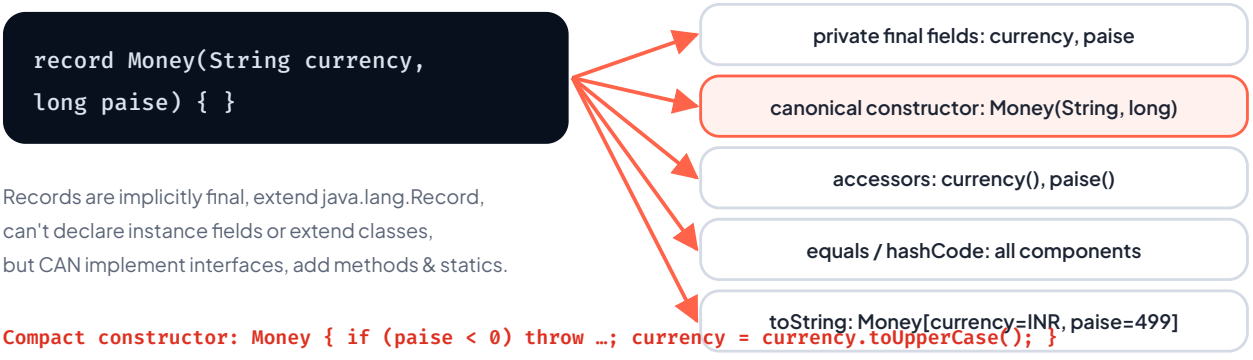
    Money {                                                        // compact canonical
constructor
        Objects.requireNonNull(currency);
        if (paise < 0) throw new IllegalArgumentException("negative amount");
        currency = currency.toUpperCase();                         // assign parameters, not
fields
    }
    Money plus(Money o) {
        if (!currency.equals(o.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(currency, paise + o.paise);
    }
    public int compareTo(Money o) { return Long.compare(paise, o.paise); }
    @Override public String toString() { return currency + " " + paise / 100 + "." +
String.format("%02d", paise % 100); }
}
Money m = new Money("inr", 49_900).plus(new Money("INR", 100));
System.out.println(m + " | equal: " + m.equals(new Money("INR", 50_000)) + " | accessor: " +
m.paise());
```


© TechTribers. For personal use only. Do not redistribute.

```
try { new Money("INR", -1); } catch (IllegalArgumentException e) {
System.out.println(e.getMessage()); }
```

▶ Output

```
INR 500.00 | equal: true | accessor: 50000
negative amount
```



⚠️ **Interview trap**

Accessors are `paise()`, not `getPaise()`. Libraries or templates that expect JavaBean getters won't find them without configuration.

📋 **Q364** MJ-025 • Intermediate

What is the difference between `Stream.toList()` and `collect(Collectors.toList())`?

⚡ **Quick answer**

`Stream.toList()` (Java 16) returns an **unmodifiable** list and allows `null` elements. `Collectors.toList()` makes no guarantees about type or mutability (in practice a mutable `ArrayList`), and `Collectors.toUnmodifiableList()` is unmodifiable but rejects nulls.

Java

```
List<String> a = Stream.of("x", "y").toList();
List<String> b = Stream.of("x", "y").collect(Collectors.toList());

try { a.add("z"); } catch (UnsupportedOperationException e) { System.out.println("toList():
unmodifiable"); }
b.add("z");
System.out.println("Collectors.toList(): " + b);

System.out.println(Stream.of("x", null).toList());                    // nulls allowed
try { Stream.of("x", null).collect(Collectors.toUnmodifiableList()); }
catch (NullPointerException e) { System.out.println("toUnmodifiableList(): rejects null"); }
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
toList(): unmodifiable
Collectors.toList(): [x, y, z]
[x, null]
toUnmodifiableList(): rejects null
```

⚠️ Interview trap

Replacing `collect(Collectors.toList())` with `toList()` across a codebase is not a pure refactor: any caller that later adds to or sorts the list will now throw `UnsupportedOperationException`.

🔐 Q365 MJ-026 • Advanced

What changed with strong encapsulation of JDK internals in Java 16 and 17?

⚡ Quick answer

Java 9–15 allowed illegal reflective access to JDK internals with a warning. Java 16 (JEP 396) denied it by default and Java 17 (JEP 403) removed the `--illegal-access` escape hatch entirely. Only critical APIs such as `sun.misc.Unsafe` remain accessible; anything else needs explicit `--add-opens`.

Why the JDK did it: internals change between releases, and libraries reaching into them blocked JDK evolution and created security risks. Replacements exist for most use cases: `VarHandle`, `MethodHandles.privateLookupIn`, the FFM API, and the Class-File API.

Terminal

```
// Typical failure after upgrading to 17:
// java.lang.reflect.InaccessibleObjectException: Unable to make field private final byte[]
//   java.lang.String.value accessible: module java.base does not "opens java.lang" to unnamed
module

// 1. Best: upgrade the library that does this (Lombok, Mockito, Spring, Kryo, old Gson...)
// 2. Workaround, documented and temporary:
//    java --add-opens java.base/java.lang=ALL-UNNAMED -jar app.jar
// 3. Or in the JAR manifest:  Add-Opens: java.base/java.lang
```

⚠️ Interview trap

Adding a dozen `--add-opens` flags to make a build pass hides real upgrade work. Each one is technical debt that may break again on the next JDK.

© TechTribers. For personal use only. Do not redistribute.

Q366 MJ-027

• Intermediate

What is `jpackage`, and how is it different from `jlink`?

⚡ Quick answer

`jlink` (Java 9) builds a trimmed Java **runtime** containing only the modules your app needs. `jpackage` (final in Java 16) goes further and creates a native **installer** (`.msi`, `.exe`, `.pkg`, `.dmg`, `.deb`, `.rpm`) that bundles the app with such a runtime.

Terminal

```
# Minimal runtime for a service (e.g. for a small Docker image)
jlink --add-modules java.base,java.net.http,java.sql,jdk.crypto.ec \
      --strip-debug --no-header-files --no-man-pages --compress=zip-6 \
      --output build/runtime

# Native desktop installer with a bundled runtime
jpackage --name TechTribersApp --input target/libs --main-jar app.jar \
         --main-class com.tt.Main --type deb --app-version 1.0.0
```

💡 Pro tip

`jlink` runtimes often shrink a container's Java layer from ~200 MB to ~50–70 MB. Use `jdeps --print-module-deps app.jar` to discover the module list.

Q367 MJ-028

• Advanced

What are Unix-domain socket channels (Java 16)?

⚡ Quick answer

Java 16 added `UnixDomainSocketAddress` support to `SocketChannel` and `ServerSocketChannel`, so processes on the same host can talk through a socket **file** instead of TCP on localhost. It's faster and is secured with file-system permissions.

Use cases: sidecars and local agents (Envoy, Docker's API at `/var/run/docker.sock`), talking to a local database, and container-to-container communication via a shared volume, all without opening network ports.

Java

```
Path socketFile = Files.createTempDirectory("tt").resolve("app.sock");
var address = UnixDomainSocketAddress.of(socketFile);

try (ServerSocketChannel server = ServerSocketChannel.open(StandardProtocolFamily.UNIX)) {
    server.bind(address);
    try (SocketChannel client = SocketChannel.open(address);
         SocketChannel accepted = server.accept()) {
        client.write(ByteBuffer.wrap("ping".getBytes(StandardCharsets.UTF_8)));
        ByteBuffer buf = ByteBuffer.allocate(16);
```


© TechTribers. For personal use only. Do not redistribute.

```
        accepted.read(buf);
        System.out.println("server got: " + new String(buf.array(), 0, buf.position(),
StandardCharsets.UTF_8));
    }
}
```

▶ Output

```
server got: ping
```

Section 05

Java 17 & the 8 → 17 Leap

Sealed classes, the new random-number API, deserialization filters and the summary question every interview asks.

Questions 368–372 · 5 questions



Q368 MJ-029 • Advanced

What are sealed classes and interfaces, and what are their rules?

⚡ Quick answer

A `sealed` class or interface (final in Java 17) lists exactly which types may extend it with `permits`. Each permitted subtype must be `final`, `sealed` or `non-sealed`, and must be in the same module (or the same package in the unnamed module). It models closed domains such as payment types or API results.

When all permitted subclasses are in the same file, `permits` can be omitted; the compiler infers them. Sealed types plus records give Java **algebraic data types**: sum types (`Ok | Err`) of product types (records).

Java

```
sealed interface Result<T> permits Ok, Err { }
record Ok<T>(T value) implements Result<T> { }
record Err<T>(String message, int code) implements Result<T> { }

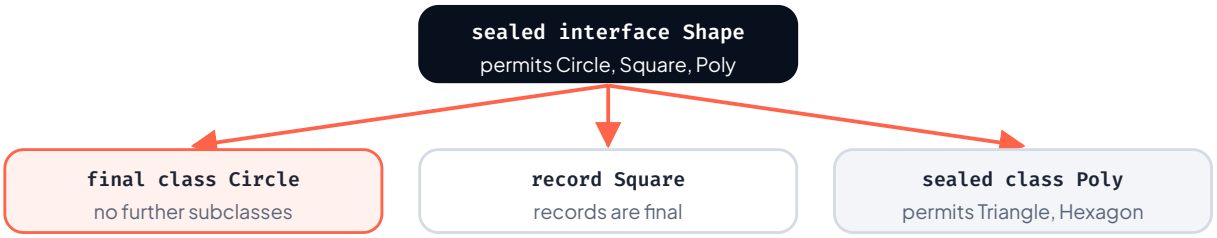
static <T> String render(Result<T> r) {
    return switch (r) {                                  // exhaustive: no default needed (Java
21)
        case Ok<T> ok   -> "200 " + ok.value();
        case Err<T> err -> err.code() + " " + err.message();
    };
}
System.out.println(render(new Ok<>("order created")));
System.out.println(render(new Err<>("insufficient stock", 409)));
System.out.println(Result.class.isSealed() + " " +
```


© TechTribers. For personal use only. Do not redistribute.

```
Arrays.toString(Arrays.stream(Result.class.getPermittedSubclasses()).map(Class::getSimpleName).toArray()));
```

▶ Output

```
200 order created
409 insufficient stock
true [Ok, Err]
```



Every permitted subclass must be final, sealed or non-sealed, and live in the same module (or package if unnamed).
Payoff (Java 21): switch over a sealed type is checked for exhaustiveness: no default, and new subtypes break the build.

💡 **Pro tip**
Use sealed hierarchies for domain results instead of throwing exceptions for expected outcomes (validation failures, "not found"): the compiler then forces every caller to handle each case.

Q369 MJ-030 • Intermediate

What is the RandomGenerator API in Java 17?

⚡ **Quick answer**
JEP 356 introduced the `RandomGenerator` interface family and a factory to pick algorithms by name (e.g. `L64X128MixRandom`, `Xoshiro256PlusPlus`). Legacy `Random`, `SplittableRandom` and `ThreadLocalRandom` implement it, so code can depend on the interface and switch algorithms freely.

Java

```
RandomGenerator g = RandomGenerator.of("L64X128MixRandom");
System.out.println(g.getClass().getSimpleName());

RandomGenerator seeded = RandomGeneratorFactory.of("Xoshiro256PlusPlus").create(42L);
int a = seeded.nextInt(100), b = seeded.nextInt(100);
RandomGenerator again = RandomGeneratorFactory.of("Xoshiro256PlusPlus").create(42L);
System.out.println("reproducible with seed: " + (a == again.nextInt(100) && b ==
again.nextInt(100)));

System.out.println(RandomGenerator.getDefault().ints(5, 1, 7).allMatch(x -> x >= 1 && x < 7));
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
L64X128MixRandom
reproducible with seed: true
true
```

⚠️ **Interview trap**

None of these algorithms are cryptographically secure. For tokens, OTPs, passwords and session IDs, use `SecureRandom`.

Q370 MJ-031 • Advanced

What are deserialization filters, and what did Java 17 add?

⚡ **Quick answer**

Deserialization filters (`ObjectInputFilter`, Java 9) let you allow or reject classes, graph depth, array sizes and object counts before objects are created from a stream. Java 17 (JEP 415) added **context-specific** filters via a JVM-wide filter factory, so each stream can get an appropriate filter.

Java deserialization has been the root of many remote-code-execution vulnerabilities (gadget chains in Commons Collections and others). Filters are defence in depth; the real fix is not deserializing untrusted Java-serialized data at all.

Java

```java
// JVM-wide allow-list via system property (no code change):
// java -Djdk.serialFilter='com.tt.dto.*;java.base/*;maxdepth=10;maxarray=10000;!*' -jar app.jar

ObjectInputFilter onlyDtos = ObjectInputFilter.Config.createFilter(
        "com.tt.dto.*;java.lang.*;java.util.*;maxdepth=10;!*");

byte[] bytes;
try (var bos = new ByteArrayOutputStream(); var oos = new ObjectOutputStream(bos)) {
    oos.writeObject(new java.util.Date());            // java.util.* is on the allow-list
    bytes = bos.toByteArray();
}
try (var ois = new ObjectInputStream(new ByteArrayInputStream(bytes))) {
    ois.setObjectInputFilter(onlyDtos);
    System.out.println("allowed: " + ois.readObject().getClass().getSimpleName());
}
```

▶ **Output**

```
allowed: Date
```

💡 **Pro tip**

Use an allow-list (`com.tt.dto.;!`), not a block-list: new gadget classes appear all the time, so blocking known-bad classes never ends.

© TechTribers. For personal use only. Do not redistribute.

Q371 MJ-032

• Intermediate

What are HexFormat and always-strict floating point in Java 17?

⚡ Quick answer

`java.util.HexFormat` (Java 17) formats and parses hexadecimal strings with configurable delimiters, prefixes and case, replacing hand-written helpers. JEP 306 made all floating-point arithmetic strict (IEEE 754), so the `strictfp` keyword is now redundant.

`strictfp` existed because x87 CPUs computed intermediate results with extra precision, so results could differ by platform. Modern CPUs (SSE2+) made strict semantics free, so Java 17 made them universal and the keyword now just triggers a warning.

Java

```
byte[] sha = {(byte) 0xCA, (byte) 0xFE, (byte) 0xBA, (byte) 0xBE};
System.out.println(HexFormat.of().formatHex(sha));
System.out.println(HexFormat.ofDelimiter(":").withUpperCase().formatHex(sha));
System.out.println(Arrays.toString(HexFormat.of().parseHex("0aff")));
System.out.println(HexFormat.of().toHexDigits((short) 255) + " " + HexFormat.fromHexDigits("ff"));
```

▶ Output

```
cafebabe
CA:FE:BA:BE
[10, -1]
00ff 255
```

Q372 MJ-033

• Intermediate

What are the most important changes between Java 8 and Java 17?

⚡ Quick answer

Language: `var`, switch expressions, text blocks, records, pattern matching for `instanceof`, sealed classes. Platform: modules, `jlink`, strong encapsulation, the HTTP client, G1 as default, ZGC/Shenandoah, container awareness, JFR for free, helpful NPEs, and many API additions.

Release	Headline features
9	Modules, JShell, collection factories, `Flow`, `StackWalker`, G1 default
10	`var`, `List.copyOf`, container awareness, AppCDS
11 (LTS)	HTTP Client, `String` methods, single-file launch, JFR open-sourced, Java EE/CORBA removed
12–13	`Collectors.teeing`, compact number format, switch expressions (preview)
14	Switch expressions final, helpful NPEs, records (preview)

© TechTribers. For personal use only. Do not redistribute.

15	Text blocks, ZGC and Shenandoah production-ready, hidden classes
16	Records, `instanceof` patterns, `Stream.toList()`, jpackage, internals denied by default
17 (LTS)	Sealed classes, `RandomGenerator`, strong encapsulation enforced, `strictfp` by default

> 💡 **Pro tip**
> Finish the answer with impact: "Moving our services from 8 to 17 cut boilerplate with records, removed a class of `ClassCastException`s with sealed types and patterns, and dropped p99 GC pauses after switching from CMS to G1."

Series progress

372 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 9 starts at Q373.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.